

# **HOLY ANGEL UNIVERSITY**

Department of Mathematics & Sciences  
Holy Angel University  
Angeles City

## **Shelf Design System**

A Midterm Course Requirement in Applications Development and Emerging  
Technologies

2<sup>nd</sup> Semester AY2026-2027



In Partial Fulfillment  
of the Requirements for the Degree  
School of Computing

Submitted to:  
**PROF. STOLK, TJAKOEN A.**  
6ADET Instructor

Submitted by:  
**EVANGELISTA, VINCE JERWYN E.**

CS301  
September 2026

## Cover and Principles

This opening page defines the Shelf design system as a shared guide for creating consistent product experiences. Its three principles set the direction: physical clarity, visual hierarchy, and semantic meaning.

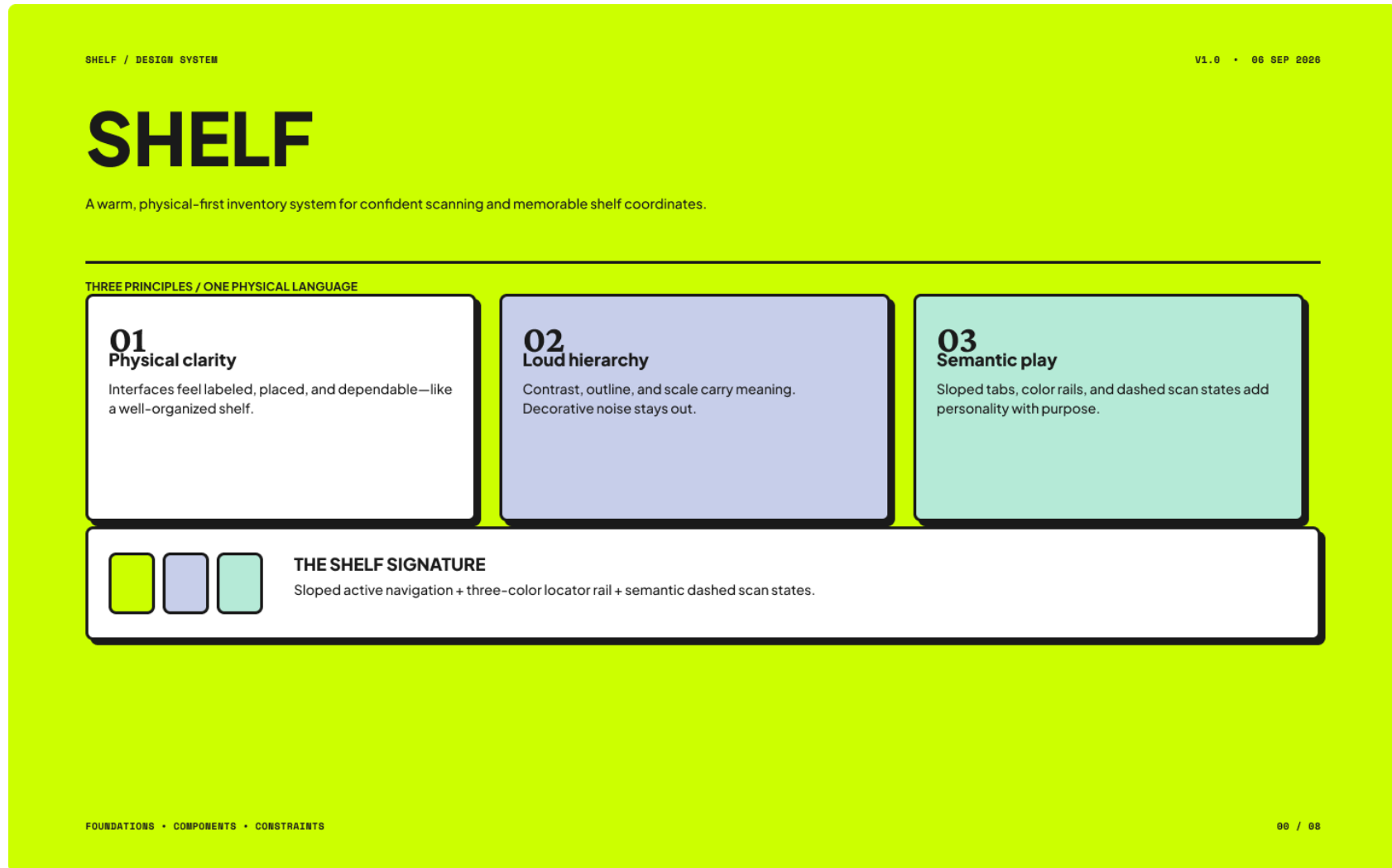


Figure 1. Shelf design system — Cover and Principles.

## Brand and Logo

The brand and logo rules protect recognition across every touchpoint. Use the approved master mark, preserve clear space and minimum size, and apply the primary, secondary, and tertiary colors consistently.

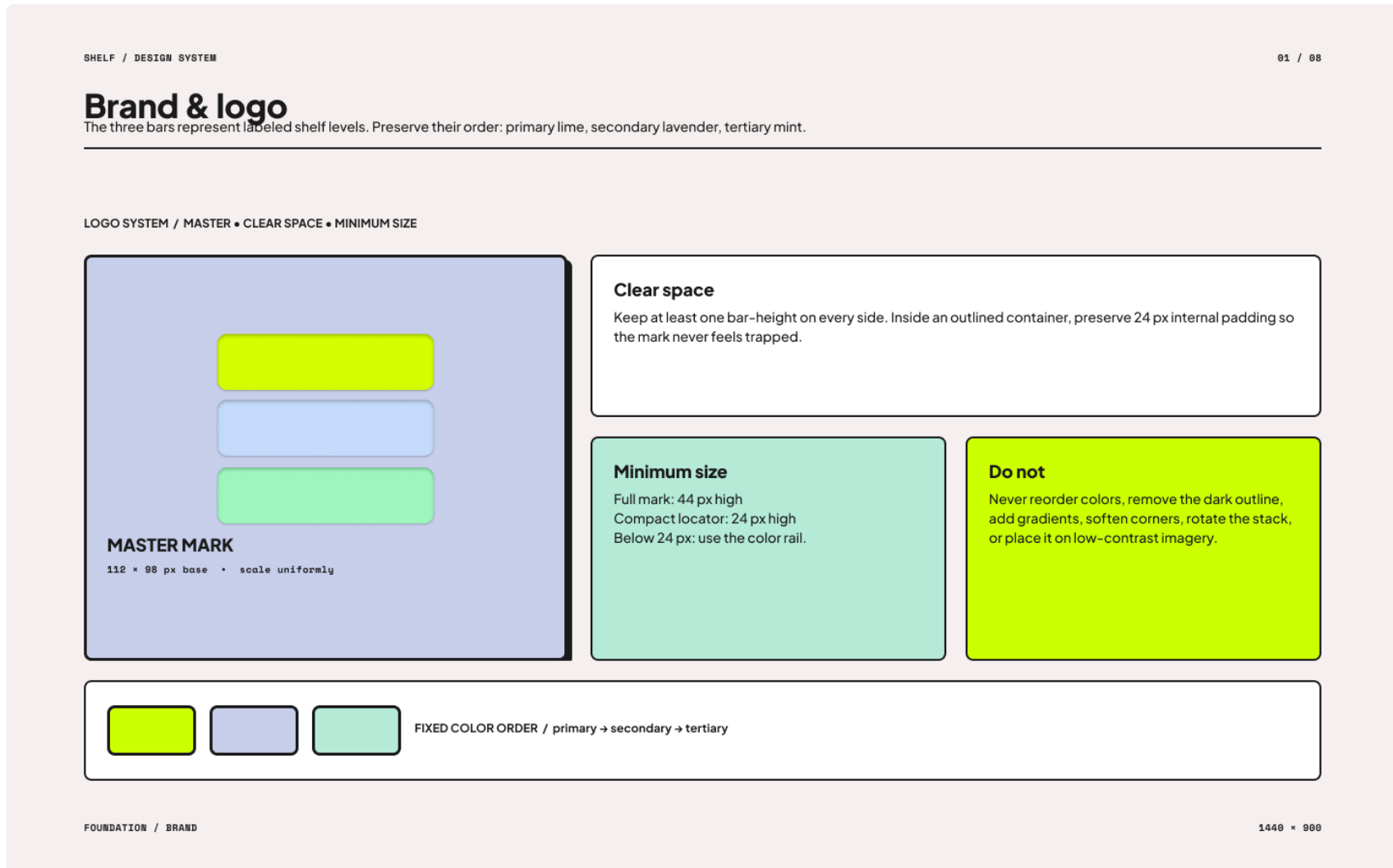


Figure 2. Shelf design system — Brand and Logo.

## Color

The color system assigns a purpose to each palette: primary actions, secondary information, success states, neutral text, and supporting surfaces. Approved contrast pairings keep text and controls legible.

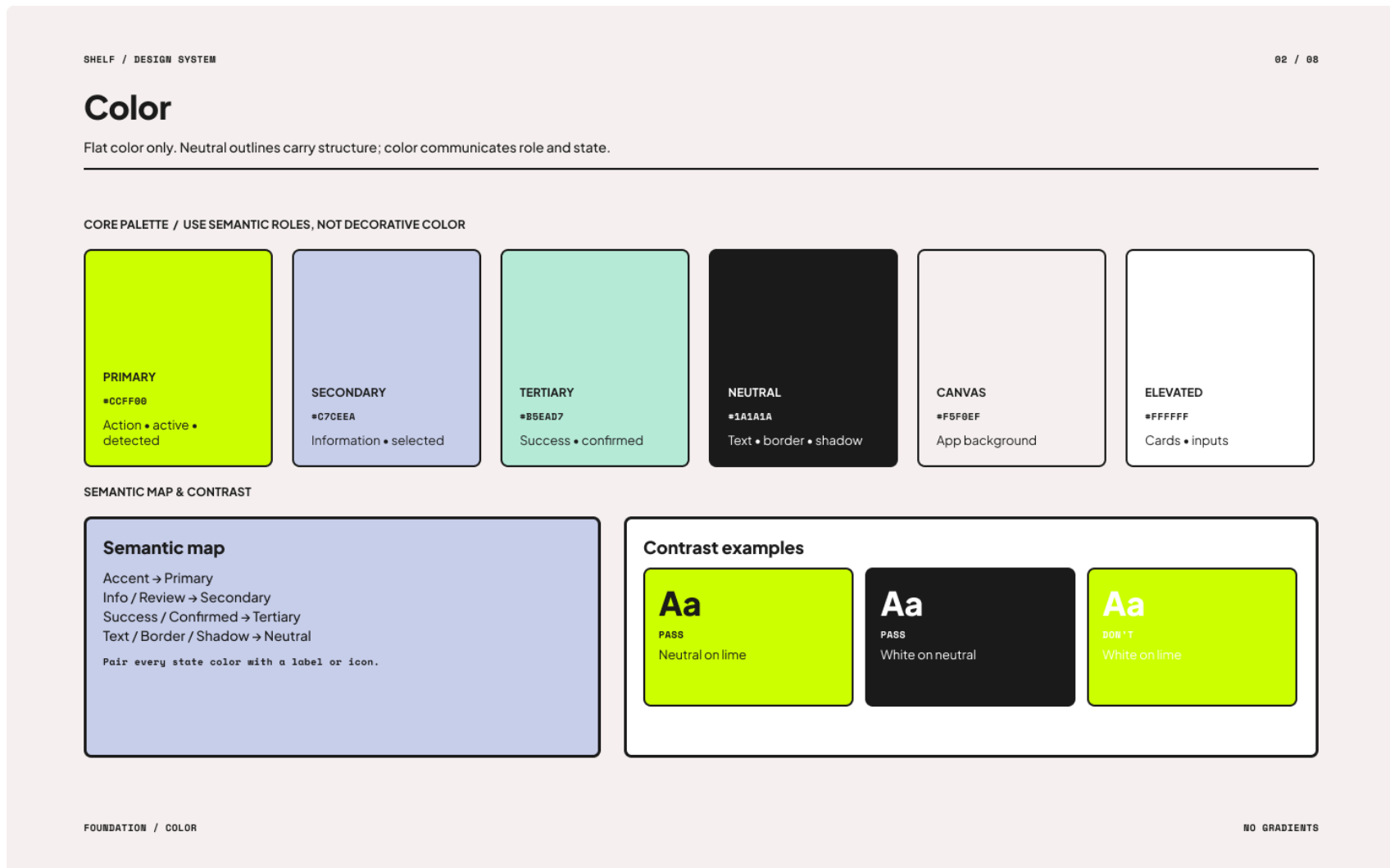


Figure 3. Shelf design system — Color.

## ColorScheme in Flutter

Shelf uses a hand-written ColorScheme rather than ColorScheme.fromSeed because the established lime, lavender, and mint colors already communicate action, review, and confirmed states. The scheme preserves those named semantic roles and keeps contrast decisions in one theme file.

```
const shelfScheme = ColorScheme(  
  brightness: Brightness.light,  
  primary: Color(0xFFCCFF00), onPrimary: Color(0xFF1A1A1A),  
  primaryContainer: Color(0xFFE5FF8A), onPrimaryContainer: Color(0xFF1A1A1A),  
  secondary: Color(0xFFC7CEEA), onSecondary: Color(0xFF1A1A1A),  
  secondaryContainer: Color(0xFFE2E7F8), onSecondaryContainer: Color(0xFF1A1A1A),  
  tertiary: Color(0xFFB5EAD7), onTertiary: Color(0xFF1A1A1A),  
  tertiaryContainer: Color(0xFFDDF8EF), onTertiaryContainer: Color(0xFF1A1A1A),  
  error: Color(0xFFBA1A1A), onError: Color(0xFFFFFFFF),  
  errorContainer: Color(0xFFFFDAD6), onErrorContainer: Color(0xFF410002),  
  surface: Color(0xFFFFFFFF), onSurface: Color(0xFF1A1A1A),  
  onSurfaceVariant: Color(0xFF373737), outline: Color(0xFF707070),  
  outlineVariant: Color(0xFFC7CEEA), shadow: Color(0xFF1A1A1A),  
  scrim: Color(0xFF000000), inverseSurface: Color(0xFF1A1A1A),  
  onInverseSurface: Color(0xFFFFFFFF), inversePrimary: Color(0xFFCCFF00),  
  surfaceTint: Color(0xFFCCFF00),  
);
```

| Role      | Hex     | Used for                               |
|-----------|---------|----------------------------------------|
| primary   | #CCFF00 | Main buttons, active state, detections |
| onPrimary | #1A1A1A | Text and icons on primary              |
| secondary | #C7CEEA | Review and selected state              |
| surface   | #FFFFFF | Cards, sheets, and dialogs             |
| onSurface | #1A1A1A | Body text, icons, and borders          |
| error     | #BA1A1A | Validation and destructive actions     |

Contrast: #1A1A1A on #F5F0EF, #FFFFFF, and #CCFF00 is high contrast. Dark mode: light only; widgets use shelfScheme or Theme.of(context).colorScheme.

## Typography

Typography establishes hierarchy and readability. Primary, utility, and brand/editorial roles define where each type style belongs, while the scale and weights keep headings, body copy, and supporting text consistent.

SHELF / DESIGN SYSTEM

03 / 08

# Typography

Three families, three jobs. Interface clarity always wins over novelty.

---

### FAMILY ROLES

**PLUS JAKARTA SANS**  
**Primary UI**  
Headings, body, controls, navigation

**SPACE MONO**  
**Utility**  
Coordinates, scan metadata, technical labels

**CRIMSON PRO**  
**Brand / editorial**  
Rare expressive moments only

### TYPE SCALE / STYLE • WEIGHT • SIZE / LINE

BRAND / DISPLAY • CRIMSON PRO BOLD • 40 / 44

## Everything has a place.

## Scan it. Find it. Put it back.

Shelf uses clear labels and physical coordinates so users can retrieve items with confidence.

SPACE A / CABINET 02 / TOP SHELF

### Published styles

| DISPLAY & HEADINGS |           |    |      | BODY & UTILITY   |          |    |      |
|--------------------|-----------|----|------|------------------|----------|----|------|
| UI/Display/Large   | ExtraBold | 34 | / 40 | UI/Body/Large    | Regular  | 16 | / 24 |
| UI/Display/Medium  | ExtraBold | 28 | / 35 | UI/Body/Default  | Medium   | 14 | / 20 |
| UI/Heading/Large   | ExtraBold | 23 | / 29 | UI/Body/Small    | Regular  | 13 | / 16 |
| UI/Heading/Medium  | ExtraBold | 18 | / 23 | UI/Label/Default | Bold     | 12 | / 15 |
| UI/Heading/Small   | Bold      | 16 | / 20 | Navigation/Label | SemiBold | 11 | / 14 |
|                    |           |    |      | UI/Caption       | Medium   | 10 | / 13 |
|                    |           |    |      | UI/Micro         | Bold     | 9  | / 11 |

CONSTRAINT

Minimum body size is 13 px. Limit headings to two lines. Use uppercase only for short utility labels. Never use Crimson Pro or Space Mono in form fields, buttons, or paragraphs.

FOUNDATION / TYPOGRAPHY

14 STYLES

Figure 4. Shelf design system — Typography.

## TextTheme in Flutter

Plus Jakarta Sans remains the primary interface family from the design system. Space Mono is reserved for compact metadata such as coordinates and scan labels. Both are supplied through the `google_fonts` package and are assigned once in the theme.

```
final shelfTextTheme = TextTheme(  
  headlineSmall: GoogleFonts.plusJakartaSans(  
    fontSize: 28, fontWeight: FontWeight.w800, color: Color(0xFF1A1A1A)),  
  bodyMedium: GoogleFonts.plusJakartaSans(  
    fontSize: 16, fontWeight: FontWeight.w400, color: Color(0xFF1A1A1A)),  
  labelSmall: GoogleFonts.spaceMono(  
    fontSize: 12, fontWeight: FontWeight.w500, color: Color(0xFF373737)),  
  labelLarge: GoogleFonts.plusJakartaSans(  
    fontSize: 14, fontWeight: FontWeight.w700, color: Color(0xFF1A1A1A)),  
);
```

| Your style   | Flutter slot  | Size | Weight | Used for                                          |
|--------------|---------------|------|--------|---------------------------------------------------|
| Heading      | headlineSmall | 28   | 800    | Screen titles and primary page headings           |
| Body         | bodyMedium    | 16   | 400    | Descriptions, form content, and item details      |
| Caption      | labelSmall    | 12   | 500    | Timestamps, coordinates, hints, and scan metadata |
| Action label | labelLarge    | 14   | 700    | Filled buttons, tabs, and important controls      |

```
Text('Shelf', style: Theme.of(context).textTheme.headlineSmall)
```

## Spacing and Geometry

Spacing and geometry turn layout decisions into repeatable rules. Use the foundational spacing steps, standard radii, and component outlines so related elements align and controls retain a consistent shape.

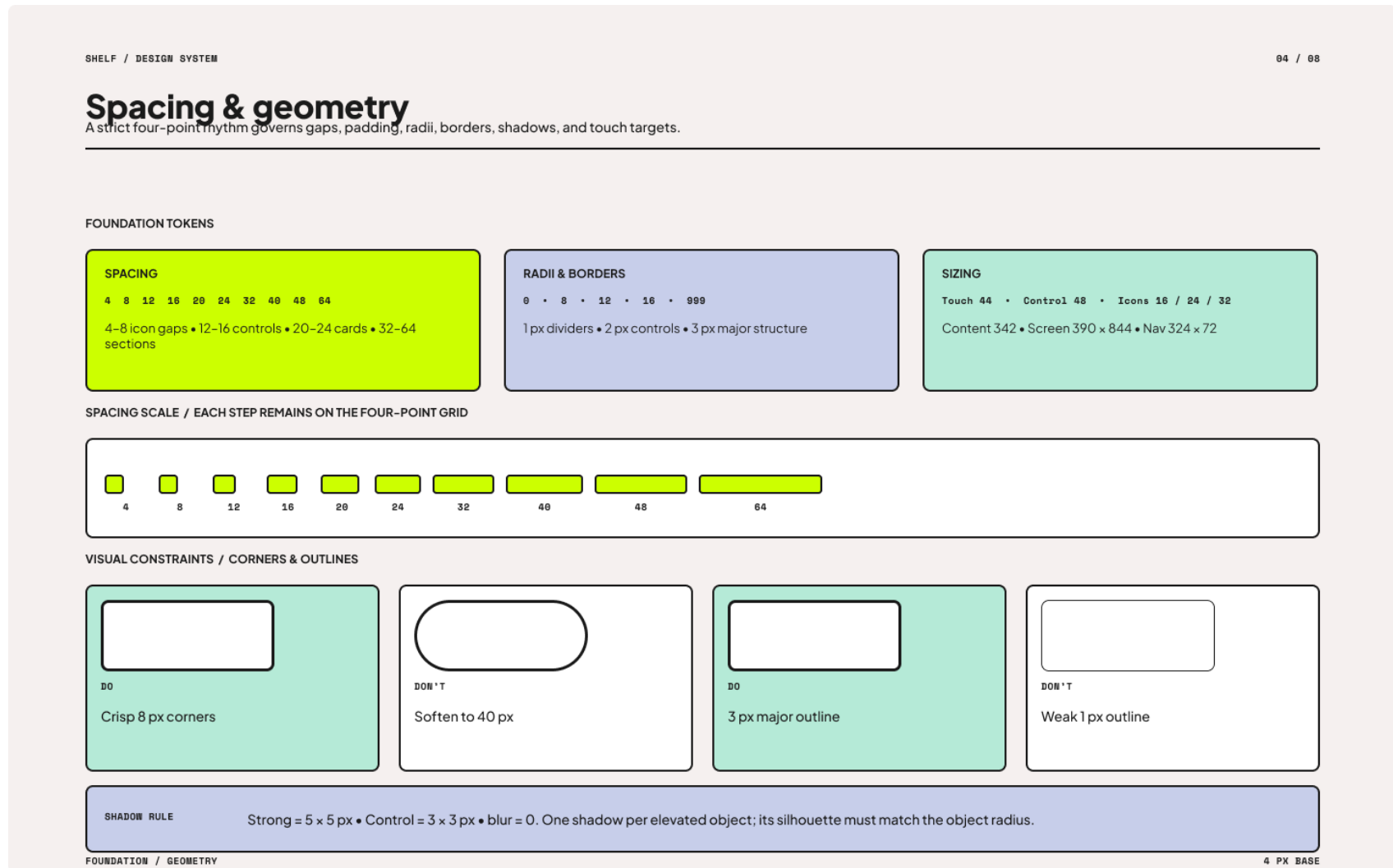


Figure 5. Shelf design system — Spacing and Geometry.

## Spacing Constants in Flutter

Shelf keeps the four-point rhythm already defined in the design system. Tokens name common layout decisions so that screen spacing is consistent and can be changed without searching every widget.

```
class AppSpacing {  
  static const double xs = 4;  
  static const double sm = 8;  
  static const double listItem = 12;  
  static const double md = 16;  
  static const double lg = 24;  
  static const double section = 32;  
}
```

| Decision               | Value | Token / use                                                 |
|------------------------|-------|-------------------------------------------------------------|
| Base unit              | 4 px  | AppSpacing.xs; all measurements stay on the four-point grid |
| Screen edge padding    | 24 px | AppSpacing.lg; the mobile layout rail                       |
| Gap between list items | 12 px | AppSpacing.listItem; compact card or section lists          |
| Gap between sections   | 32 px | AppSpacing.section; separates major content groups          |

```
padding: const EdgeInsets.all(AppSpacing.md)
```

## Layout and Constraints

Layout rules define responsive behavior before screens are built. Follow the reference frame, width constraints, and visual examples to preserve clear hierarchy and readable content across screen sizes.

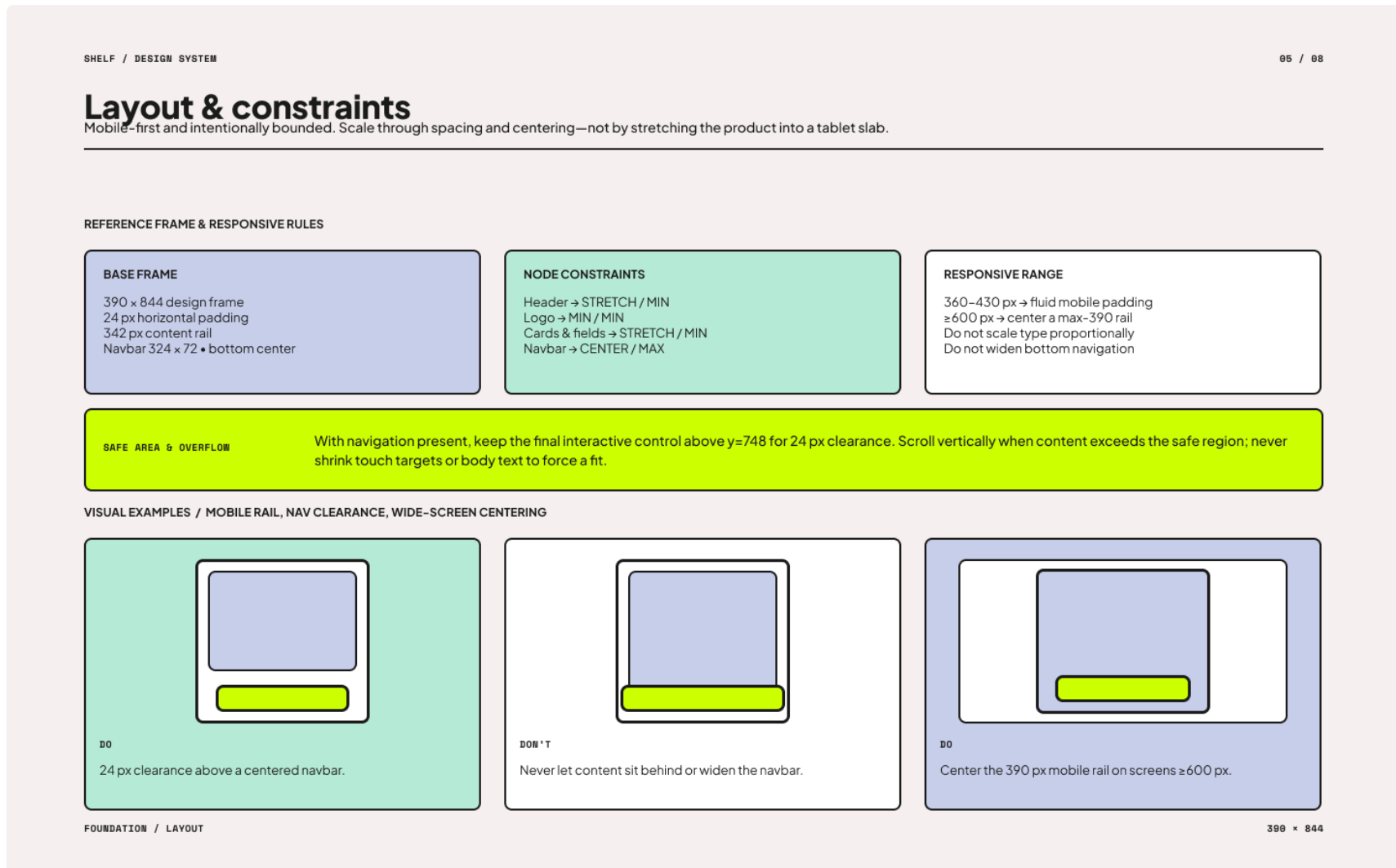


Figure 6. Shelf design system — Layout and Constraints.

## Components

Components are reusable interface building blocks, including buttons, inputs, navigation, cards, tags, and local navigation. Their defined states and placement rules make the product faster to build and easier to learn.

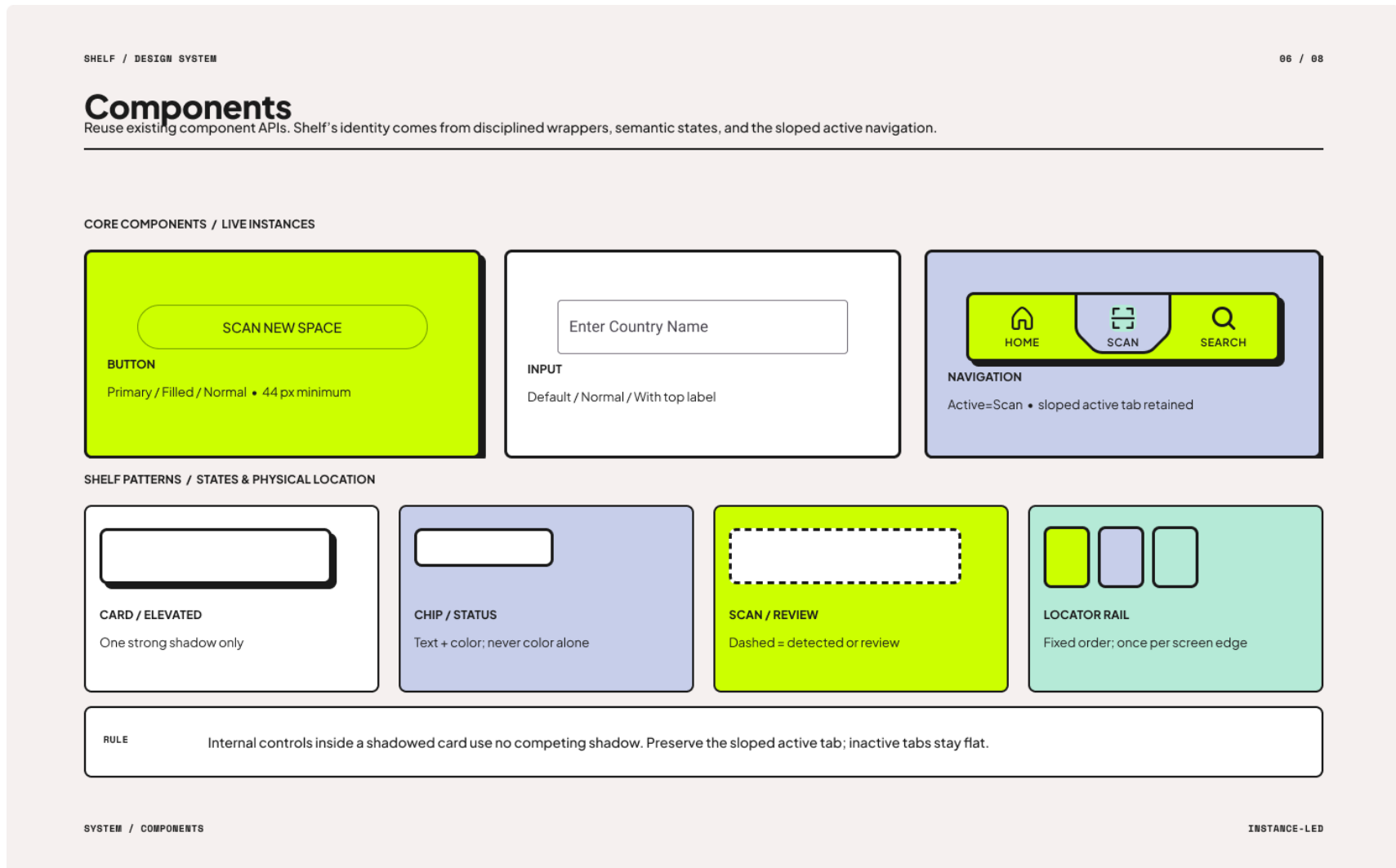


Figure 7. Shelf design system — Components.

## Component Files and Contracts

The existing visual components become reusable Flutter widgets with explicit inputs and callbacks. Each component renders the data it receives and reports interactions upward; it does not own page state or call `setState`.

| Component             | File                                     | Constructor parameters                                                                                                | Appears on                                                 |
|-----------------------|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| PrimaryActionButton   | lib/widgets/primary_action_button.dart   | String label, IconData? icon, VoidCallback? onPressed                                                                 | Welcome, setup, scan, loan and return flows                |
| ShelfTextField        | lib/widgets/shelf_text_field.dart        | String label, TextEditingController controller, String? Function(String?)? validator, ValueChanged<String>? onChanged | Workspace, container, section, and item forms              |
| ShelfBottomNavigation | lib/widgets/shelf_bottom_navigation.dart | int currentIndex, ValueChanged<int> onDestinationSelected                                                             | Home, Spaces, Search, and Activity shell                   |
| SectionCard           | lib/widgets/section_card.dart            | Section section, int itemCount, VoidCallback onTap                                                                    | Container setup, container overview, and section inventory |
| ItemLocationCard      | lib/widgets/item_location_card.dart      | Item item, VoidCallback onTap                                                                                         | Search results, section inventory, and checkout selection  |
| StatusChip            | lib/widgets/status_chip.dart             | String label, ItemStatus status                                                                                       | Item detail, cards, loan history, and scan review          |

| Component         | File                                 | Constructor parameters                                                                              | Appears on                                                  |
|-------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| ScanCandidateTile | lib/widgets/scan_candidate_tile.dart | ScanCandidate candidate,<br>VoidCallback onAccept,<br>VoidCallback onEdit,<br>VoidCallback onRemove | Section scan review<br>and correction flow                  |
| EmptyState        | lib/widgets/empty_state.dart         | String title, String<br>message, IconData icon,<br>String? actionLabel,<br>VoidCallback? onAction   | Empty search, section,<br>activity, and workspace<br>states |

Implementation rules: component constructors are const wherever their fields allow it, and reusable widgets receive models, values, and callbacks rather than a list or setState from the parent page.

## Theme File

The palette and text scale are assembled in lib/theme.dart. It is the only application file that contains literal color values.

```
// lib/theme.dart
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

final appTheme = ThemeData(
  useMaterial3: true, colorScheme: shelfScheme, textTheme: shelfTextTheme,
  scaffoldBackgroundColor: const Color(0xFFFF5F0EF),
  cardTheme: const CardThemeData(
    color: Color(0xFFFFFFFF), margin: EdgeInsets.zero, elevation: 0,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.all(Radius.circular(8)),
      side: BorderSide(color: Color(0xFF1A1A1A), width: 3),
    ),
  ),
  filledButtonTheme: FilledButtonThemeData(
    style: FilledButton.styleFrom(minimumSize: const Size.fromHeight(48)),
  ),
);
```

## Accessibility and Content

Accessibility and content guidance ensures the interface remains understandable and usable. Check contrast, focus states, readable labels, helpful feedback, and concise language before release.

SHELF / DESIGN SYSTEM

07 / 08

# Accessibility & content

Neobrutalist contrast should increase clarity, not visual noise. Every state must remain understandable without color.

CORE RULES

**ACCESSIBILITY**

- 44 px minimum touch target
- 8 px minimum target separation
- Visible 3 px focus treatment
- Never encode status by color alone

**CONTENT**

- Use sentence case for screens
- Begin actions with a verb
- Name physical locations consistently
- Keep errors specific and recoverable

**SCAN STATES**

- Solid border = confirmed
- Dashed 7 / 5 = detected or review
- Lime = active • Mint = successful
- Pair every state with plain-language text

VISUAL EXAMPLES / SIZE, STATUS, AND PHYSICAL LANGUAGE



44 px

28 px

**DO / 44 PX MINIMUM**

Preserve the target even when the visible icon is smaller.



**DO / ICON + TEXT + COLOR**

The message still works in grayscale.



**DO** Top shelf

**DON'T** Shelf 1

**PREFER PHYSICAL WORDS**

Use Space → Container → Section → Item.

**MOTION & FEEDBACK**

Use instant state changes or 120–180 ms translations; no springy overshoot. Pressed controls may shift 3 px toward their hard shadow. Respect reduced motion and always pair loading or scanning with text.

SYSTEM / ACCESSIBILITY

CLARITY FIRST

Figure 8. Shelf design system — Accessibility and Content.

## Handoff Checklist

The handoff checklist turns the design system into a practical quality gate. It confirms that structure, semantics, accessibility, and developer readiness are aligned before implementation begins.

SHELF / DESIGN SYSTEM

08 / 08

# Handoff checklist

A screen is ready only when structure, semantics, accessibility, and physical-location language agree.

DEFINITION OF DONE

01STRUCTURE

02SEMANTICS

03ACCESSIBILITY

04QA

RELEASE CHECKLIST

### DESIGN QA

- ☐ Uses Shelf / Core variables
- ☐ Uses published text and effect styles
- ☐ 3 px major outlines are consistent
- ☐ Shadows stay inside parent bounds
- ☐ Navigation keeps 24 px content clearance
- ☐ Empty, loading, success, and error states exist

### CONTENT QA

- ☐ Locations follow Space → Container → Section → Item
- ☐ Actions begin with verbs
- ☐ Scan state includes plain-language feedback
- ☐ Labels do not rely on color alone
- ☐ Long names wrap or truncate intentionally
- ☐ No placeholder or lorem ipsum remains

### DEVELOPER QA

- ☐ 390 x 844 reference matches
- ☐ 360–430 mobile range tested
- ☐ 44 px minimum targets preserved
- ☐ Reduced motion is respected
- ☐ Variables appear in Dev Mode syntax
- ☐ Focus, keyboard, and screen-reader order tested

### SHELF SIGNATURE

Sloped active navigation + three-color locator rail + semantic dashed scan states. Use consistently and sparingly.

SYSTEM / HANDOFF

READY TO BUILD

Figure 9. Shelf design system — Handoff Checklist.

## What Changed and Why

| Element        | Prelim said                                                        | Now says                                                           | Why it changed                                                        |
|----------------|--------------------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------|
| Palette        | Six swatches and contrast examples.                                | Custom ColorScheme; no widget-level colors.                        | Maps the established Shelf colors to Material roles.                  |
| Typography     | Three type families and fourteen visual styles.                    | TextTheme names heading, body, caption, and action styles.         | Widgets use theme slots, not copied font sizes.                       |
| Spacing        | Four-point grid from 4 to 64 px.                                   | AppSpacing tokens for base, list, edge, and section gaps.          | Turns visual values into reusable Flutter code.                       |
| Components     | Visual button, input, navigation, card, chip, and review patterns. | Widget files, parameters, and screen use cases for repeated parts. | Defines buildable APIs; values and callbacks stay outside page state. |
| Theme assembly | Palette and type documented separately.                            | lib/theme.dart assembles color, type, card, and button rules.      | Prevents hardcoded styles as screens grow.                            |
| Dark mode      | No implementation decision.                                        | Light only; colors always come from the theme.                     | Avoids a later partial dark mode with hardcoded colors.               |